DEVOPS RUNBOOK

RelayMart

DevOps lifecycle
from the clone to live monitoring

This is the procedure for the RelayMart services in the APP folder. Clone the source, prove it with Docker Compose, push one image per service to ECR, run those same images on ECS and on EKS, and read the logs in Kibana.

Do the phases in order. Do not open ECS until Compose and Kibana have already shown a register and an OTP.

The ten phases

- 1 Get the source onto the server
- 2 See how frontend, API, and backend are split
- 3 Start the stack with Docker Compose
- 4 Prove register, email OTP, mobile OTP, login, tracking, and checkout
- 5 Watch those events in Kibana, Grafana, and RabbitMQ
- 6 Build one Docker image per service
- 7 Push every image to Amazon ECR
- 8 Put Postgres and Redis on AWS (RDS and ElastiCache)
- 9 Run the images on ECS, then run the same images on EKS
- 10 Stream logs with EFK and confirm monitoring

1. What you are deploying

The browser opens the frontend. The frontend calls only the API gateway. The gateway calls the backend services. Services publish events to RabbitMQ. Every process writes JSON to stdout. Fluent Bit reads those lines into Elasticsearch. Kibana is where you read them.

Layer	Service	Port	Image
Frontend	frontend	4173	relaymart/frontend
API	gateway	4000	relaymart/gateway
Backend	catalog search pricing inventory cart order	4001-4006	one image each
Backend	payment	4007	relaymart/payment
Backend	shipping review user notification recommendation	4008-4012	one image each
Backend	auth register and login	4013	relaymart/auth
Backend	verification email and mobile OTP	4014	relaymart/verification
Backend	tracking	4015	relaymart/tracking
Data	Postgres	5432	users, payments, shipments
Data	Redis (ElastiCache on AWS)	6379	sessions and OTP
Data	RabbitMQ	5672	events between services
Logs	Fluent Bit, Elasticsearch, Kibana	5601	index relaymart-*

Metrics	Prometheus and Grafana	9090 / 3000	scrapes /metrics

RabbitMQ routing keys: user.registered, otp.email, otp.mobile, payment.authorized, tracking.updated.

Honest limits. Payment writes an authorized row. It does not call a bank. Email and mobile codes are not sent by SES or SNS. They sit in Redis for five minutes, go out on RabbitMQ, and are printed in the verification log so Kibana can show them. Elasticsearch here is one node with security off. Use Amazon OpenSearch in production.

2. What to install first

On the Compose machine:

- Docker Engine 24 or newer, with docker compose version.
- The unzipped APP folder.
- Node.js 22 only if you want the checkout smoke test without Docker.

On the deploy machine, also:

- AWS CLI v2. aws sts get-caller-identity must succeed.
- Terraform 1.6 or newer, for ECS.
- kubectl pointed at your EKS cluster, for the EKS phase only.

You do not need AWS for the Compose test. You need it from the ECR phase onward: a VPC, two subnets, and an ECS task execution role that can pull from ECR and write CloudWatch logs.

3. Get the source

Step 1. Copy the project onto the server

Unzip RelayMart-source.zip. Work inside APP. Every command below assumes that directory unless it says otherwise.

```
cd APP
ls
```

Expected: You see docker-compose.yml, frontend, gateway, services, db, observability, deploy, and scripts.

Step 2. Confirm one Dockerfile per service

Each process is its own image. That file is the build contract.

```
ls frontend/Dockerfile gateway/Dockerfile
ls services/auth/Dockerfile services/verification/Dockerfile
ls services/payment/Dockerfile services/tracking/Dockerfile
```

Expected: Each path exists. scripts/ecr-push.sh builds all of them.

Folders you use later:

- db/init.sql creates users, payments, shipments, and tracking_events. Compose applies it. RDS does not.
- observability/fluent-bit is the Compose log shipper.
- deploy/ecs/main.tf is the Fargate mesh. deploy/ecs/elasticache.tf is optional RDS and ElastiCache.
- deploy/eks/mesh.yaml is the Kubernetes mesh. deploy/eks/efk.yaml is Elasticsearch, Kibana, and Fluent Bit.
- docs/RelayMart-DevOps-Lifecycle.pdf is this runbook.

4. Start it with Docker Compose

Step 3. Build and start every container

The first start is slow. Each image runs npm install, and Elasticsearch needs memory.

```
docker compose up --build
```

After the first good start you can detach:

```
docker compose up --build -d
```

Step 4. Wait until the app containers are healthy

Do not call the API while auth or elasticsearch is still starting.

```
docker compose ps
```

Expected: frontend, gateway, the backends, postgres, redis, and rabbitmq are healthy. Elasticsearch can take a minute. Kibana waits for it.

If auth, verification, or tracking is restarting:

```
docker compose logs --tail 80 auth verification tracking
```

Addresses on the server

What	URL	Login
Shop UI	http://127.0.0.1:4173	none
API gateway	http://127.0.0.1:4000	none
Kibana	http://127.0.0.1:5601	security off
RabbitMQ	http://127.0.0.1:15672	relay / relay
Grafana	http://127.0.0.1:3000	admin / see compose file
Prometheus	http://127.0.0.1:9090	none

If you are on a laptop and the stack is on a server, tunnel the ports with ssh -L.

5. Prove the application

This is the gate. If these calls fail, stop. Do not push images and do not apply Terraform. Use a new email if you register twice.

Step 5. Register a shopper

Auth writes the user in Postgres and publishes user.registered.

```
curl -s -X POST http://127.0.0.1:4000/api/register \
  -H 'content-type: application/json' \
  -d '{"name":"Asha Menon","email":"asha@example.com","mobile":"9845012345","password"
  :"Relaymart123"}'
```

Expected: HTTP 201 and a user id. Login is refused until both OTPs are confirmed.

Step 6. Send the email code

Verification stores a 6-digit code in Redis and prints it to stdout.

```
curl -s -X POST http://127.0.0.1:4000/api/verify/email/send \
  -H 'content-type: application/json' \
  -d '{"email":"asha@example.com"}'
docker compose logs --tail 40 verification
```

Expected: A JSON line with msg otp_issued, channel email, and code. The code expires in 300 seconds.

Step 7. Confirm the email

Replace THE_CODE with the six digits from that log.

```
curl -s -X POST http://127.0.0.1:4000/api/verify/email/confirm \
  -H 'content-type: application/json' \
  -d '{"email":"asha@example.com","code":"THE_CODE"}'
```

Expected: verified true, and account.ok true. Auth has marked the email verified in Postgres.

Step 8. Send and confirm the mobile code

Login stays blocked until this succeeds.

```
curl -s -X POST http://127.0.0.1:4000/api/verify/mobile/send \
```

```
  -H 'content-type: application/json' \
  -d '{"mobile":"9845012345"}'
docker compose logs --tail 20 verification
curl -s -X POST http://127.0.0.1:4000/api/verify/mobile/confirm \
  -H 'content-type: application/json' \
  -d '{"mobile":"9845012345","code":"THE_CODE"}'
```

Step 9. Log in

Auth checks the password and both verified flags, then stores a session in Redis.

```
curl -s -X POST http://127.0.0.1:4000/api/login \
  -H 'content-type: application/json' \
  -d '{"email":"asha@example.com","password":"Relaymart123"}'
```

Expected: A token and the user. A wrong password returns invalid_login. A missing OTP returns verification_required.

Step 10. Create a shipment and scan it

Tracking writes Postgres rows and publishes tracking.updated.

```
curl -s -X POST http://127.0.0.1:4000/api/tracking/shipments \
  -H 'content-type: application/json' \
  -d '{"orderId":"ord_demo","pincode":"560001"}'
```

Copy the id (trk_ followed by 8 characters) into the next call.

```
curl -s -X POST http://127.0.0.1:4000/api/tracking/shipments/trk_ab12cd34/scan \
  -H 'content-type: application/json' \
  -d '{"status":"picked_up","note":"hub scan"}'
```

Expected: status picked_up and two events. Allowed: created, picked_up, in_transit, out_for_delivery, delivered.

Step 11. Place an order through the gateway

This saga prices, reserves stock, gets an ETA, authorizes payment, creates the order, and notifies.

```
curl -s -X POST http://127.0.0.1:4000/api/checkout \
  -H 'content-type: application/json' \
  -d '{"lines":[{"productId":"meadow-tee","qty":1}],"method":"upi","pincode":"560001",
  "coupon":"STUDIO"}'
```

Expected: HTTP 201, an order id, and a trace of about nine hops. Coupon STUDIO takes 100 off an order from 999.

The shop page on port 4173 does the same register, verify, login, and tracking clicks. Read the OTP from the verification log, not from an inbox.

6. Monitor it locally

Local monitoring is the same pipeline you will use on EKS. Prove it here.

Step 12. Create the Kibana data view

Open http://127.0.0.1:5601. Stack Management, Data Views, Create data view.

- Name: relaymart
- Index pattern: relaymart-*
- Timestamp field: @timestamp

Open Discover. Set the time picker to the last 15 minutes. If it is empty, send one more OTP and refresh.

Step 13. Filter the log stream

Paste these into the Discover search bar.

```
service: "verification" and msg: "otp_issued"
service: "auth" and msg: "login"
service: "tracking" and msg: "scan"
service: "payment" and msg: "recorded"
service: "notification" and msg: "delivered"
```

Expected: otp_issued shows the address and the code. delivered means notification consumed the RabbitMQ message. If Discover is empty but docker compose logs shows JSON, Fluent Bit is not reading the Docker log files. See the fault list.

Step 14. Check the other consoles

- RabbitMQ on port 15672, user relay, password relay. Queues should include notification.otp.email, notification.otp.mobile, notification.tracking.updated, notification.payment.authorized, and notification.user.registered.
- Grafana on port 3000. User admin. Password is GF_SECURITY_ADMIN_PASSWORD in docker-compose.yml. Grafana reads Prometheus.
- Prometheus on port 9090. Query http_requests_total. The gateway count must have moved after your curls.

```
docker compose logs -f gateway verification auth
```

7. Build one image per service

Step 15. Build and list the images

Compose already built them if you used --build. This script tags them relaymart/<service>:local.

```
sh scripts/build-images.sh
docker images | grep relaymart
```

Expected: Seventeen images: frontend, gateway, catalog, search, pricing, inventory, cart, order, payment, shipping, review, user, notification, recommendation, auth, verification, tracking.

That image list is the only artifact that moves to AWS. ECS and EKS run these images. They do not compile the app a second way.

8. Push the images to ECR

Step 16. Create the repositories and push

The script creates relaymart/<service> if it is missing, then pushes the tag latest.

```
aws sts get-caller-identity
AWS_ACCOUNT_ID=123456789012 AWS_REGION=ap-south-1 sh scripts/ecr-push.sh
```

Replace the account and region. The script logs Docker in to ECR.

Step 17. Count the repositories

```
aws ecr describe-repositories --region ap-south-1 \
  --query 'repositories[?starts_with(repositoryName, `relaymart/`)].repositoryName' \
  --output text
```

Expected: Seventeen names, from relaymart/frontend through relaymart/tracking.

The prefix you paste into Terraform and into the EKS renderer is:

```
123456789012.dkr.ecr.ap-south-1.amazonaws.com/relaymart
```

A full image looks like that prefix, then /auth:latest.

9. Data services on AWS

Compose started Postgres, Redis, and RabbitMQ for you. ECS and EKS do not. On AWS, Redis is ElastiCache and Postgres is RDS. The broker can be Amazon MQ, or a RabbitMQ you run yourself.

Step 18. Change the RDS password first

Open deploy/ecs/elasticache.tf and replace the placeholder database password before the first apply.

Step 19. Create RDS and ElastiCache

From deploy/ecs. This apply also starts the ECS services. Auth will fail until you set the database URL.

That is expected on this pass.

```
cd deploy/ecs
terraform init
terraform apply \
  -var vpc_id=vpc-xxxx \
  -var 'subnet_ids=["subnet-a","subnet-b"]' \
  -var execution_role_arn=arn:aws:iam::123456789012:role/ecsTaskExecutionRole \
  -var image_prefix=123456789012.dkr.ecr.ap-south-1.amazonaws.com/relaymart \
  -var create_rds=true \
  -var create_elasticache=true
```

```
terraform output elasticache_endpoint
terraform output rds_endpoint
```

Step 20. Load the schema

Compose loaded db/init.sql. RDS will not.

```
psql "postgres://relay:YOUR_PASSWORD@RDS_ENDPOINT:5432/relaymart" \
  -f ../../db/init.sql
```

Expected: Tables users, payments, shipments, and tracking_events.

RabbitMQ is not created by this Terraform. Set rabbit_url on the next apply if you have a broker. Leave it empty and auth still runs, but no events are published and the notification consumer stays idle.

10. Run it on ECS

Step 21. Apply again with the real URLs

Leave elasticsearch_host empty the first time. Logs stay in CloudWatch. Set it later when the tasks can reach Elasticsearch on port 9200.

```
terraform apply \
  -var vpc_id=vpc-xxxx \
  -var 'subnet_ids=["subnet-a","subnet-b"]' \
  -var execution_role_arn=arn:aws:iam::123456789012:role/ecsTaskExecutionRole \
  -var image_prefix=123456789012.dkr.ecr.ap-south-1.amazonaws.com/relaymart \
  -var database_url='postgres://relay:YOUR_PASSWORD@RDS_ENDPOINT:5432/relaymart' \
  -var redis_url='redis://ELASTICACHE_ENDPOINT:6379' \
  -var rabbit_url='' \
  -var elasticsearch_host=''
```

Expected: Cluster relaymart. One Fargate service per process. DNS is gateway.relaymart.local, auth.relaymart.local, and the same pattern for the others. frontend and gateway get public IPs.

Step 22. Confirm the services

```
aws ecs list-services --cluster relaymart --region ap-south-1
aws ecs describe-services --cluster relaymart \
  --services auth gateway frontend --region ap-south-1 \
  --query 'services[].{name:serviceName,running:runningCount,desired:desiredCount}'
```

Expected: runningCount equals desiredCount. gateway and frontend want 2. The others want 1.

Step 23. Call the public gateway

Find the gateway task network interface, open it in EC2, and copy the public IPv4 address. Then repeat phase 5 against that IP.

```
TASK=$(aws ecs list-tasks --cluster relaymart --service-name gateway \
  --region ap-south-1 --query 'taskArns[0]' --output text)
aws ecs describe-tasks --cluster relaymart --tasks "$TASK" \
  --region ap-south-1 --query 'tasks[0].attachments[0].details'
```

curl http://THAT_IP:4000/health must return ok before you repeat register and checkout.

Step 24. Read the ECS logs

With elasticsearch_host empty, the group is /relaymart/mesh and the stream prefix is the service name.

```
aws logs tail /relaymart/mesh --region ap-south-1 --follow \
  --filter-pattern otp_issued
```

Expected: The same otp_issued JSON you saw from docker compose logs.

When elasticsearch_host is set, Terraform adds a FireLens Fluent Bit sidecar on every task and ships application logs to that host, index prefix relaymart. The sample uses TLS off. Turn TLS on for Amazon OpenSearch. CloudWatch still keeps the sidecar logs.

11. Run the same images on EKS

The node role must be able to pull from ECR. AmazonEC2ContainerRegistryReadOnly on the node role is enough. There is no imagePullSecret in this manifest.

Step 25. Render the manifests with your ECR prefix and apply them

IMAGE_PREFIX has no trailing slash and no service name.

```
cd ../..
IMAGE_PREFIX=123456789012.dkr.ecr.ap-south-1.amazonaws.com/relaymart \
  node deploy/eks/render.mjs
kubectl apply -f deploy/eks/mesh.yaml
```

Expected: Namespace relaymart. One Deployment and one Service per process. Gateway has 2 replicas and an HPA. The Ingress host is relaymart.local.

Step 26. Point the data URLs at AWS

render.mjs writes DATABASE_URL, REDIS_URL, and RABBIT_URL. The defaults are the Compose hostnames. Edit those three in deploy/eks/render.mjs to the RDS, ElastiCache, and broker endpoints. Render again and apply again.

```
kubectl -n relaymart get deploy
kubectl -n relaymart rollout status deploy/auth
kubectl -n relaymart rollout status deploy/gateway
```

Step 27. Open the gateway and repeat the proof

```
kubectl -n relaymart port-forward svc/gateway 4000:4000
```

In another terminal, repeat the phase 5 curls against http://127.0.0.1:4000. A 201 from register means the pod reached RDS. Port-forward svc/frontend 4173:4173 if you want the shop page.

12. EFK monitoring on EKS

Step 28. Install Elasticsearch, Kibana, and Fluent Bit

Elasticsearch requests 1 GB. A tiny node leaves the pod Pending.

```
kubectl apply -f deploy/eks/efk.yaml
kubectl -n logging get pods
```

Expected: elasticsearch and kibana Running. One fluent-bit pod per node, also Running.

Step 29. Open Kibana

```
kubectl -n logging port-forward svc/kibana 5601:5601
```

Create the data view relaymart-* again, timestamp @timestamp. Fluent Bit tails /var/log/containers, keeps lines that contain service, parses the JSON, and writes index relaymart-*.

Step 30. Generate traffic and confirm the live stream

With the gateway port-forward still up, send another email OTP. In Discover, filter:

```
service: "verification" and msg: "otp_issued"
```

Expected: A new line within a few seconds. That is the end of the path: the Compose proof is now ECR images

on EKS, and the log is in Kibana.

```
kubectl -n logging logs daemonset/fluent-bit --tail 30
```

Prometheus did not move with EFK. On EKS, install kube-prometheus-stack if you want the Grafana boards from port 3000. Each Deployment is already annotated prometheus.io/scrape, prometheus.io/port, and prometheus.io/path.

13. Definition of done

Check	Where	Pass
Compose healthy	docker compose ps	auth gateway postgres redis up
Register	POST /api/register	201 and a user id
Both OTPs	verification log and Kibana	otp_issued then verified
Login	POST /api/login	a token after both OTPs
Tracking	POST .../scan	picked_up stored
Checkout	POST /api/checkout	201 and a trace
ECR	describe-repositories	17 relaymart/* repos
ECS	describe-services	running equals desired
EKS	kubectl get deploy	namespace relaymart ready
Kibana	Discover relaymart-*	otp_issued from the cluster

14. When something fails

auth keeps restarting

The database URL is wrong, the task cannot reach RDS, or init.sql was never applied. This Terraform allows port 5432 and 6379 from 10.0.0.0/8 only.

Login says verification_required

Confirm email and mobile. Registering the same email again returns email_or_mobile_taken. Use a new address.

Kibana is empty but the container log has JSON

On Compose, Fluent Bit reads /var/lib/docker/containers. That is the Docker json-file driver. containerd, rootless Docker, or another log driver will not match observability/fluent-bit/fluent-bit.conf. On EKS it reads /var/log/containers, and the fluent-bit DaemonSet must be Running.

elasticsearch is Pending

It requests 1 GB. Free a node, or lower that request in deploy/eks/efk.yaml on a small cluster.

ECS cannot pull the image

image_prefix does not match the account that owns the repositories, or the execution role cannot call ecr:GetAuthorizationToken and ecr:BatchGetImage.

FireLens tasks fail

elasticsearch_host is not reachable from the task subnet on port 9200, or the cluster requires TLS and this sample sets tls Off.

15. Stop it

```
docker compose down -v
```

That deletes the Compose database volume.

```
cd deploy/ecs
terraform destroy
```

This Terraform file sets skip_final_snapshot = true. Destroy deletes the database with no snapshot. Do not point it at data you must keep.

```
kubectl delete -f deploy/eks/mesh.yaml
kubectl delete -f deploy/eks/efk.yaml
```

16. How the pieces fit

One set of images. Compose proves the images, the schema, the OTP, and the log pipeline. ECR stores them. ECS runs them on Fargate with Cloud Map. EKS runs the same tags as Deployments. Stdout stays the contract: JSON lines, shipped by Fluent Bit, read in Kibana. Grafana answers how many and how slow. Kibana answers what one register, OTP, payment, or scan did.

For production, swap the stand-ins: SES or SNS in the notification consumer, a real payment provider, and Amazon OpenSearch with TLS.